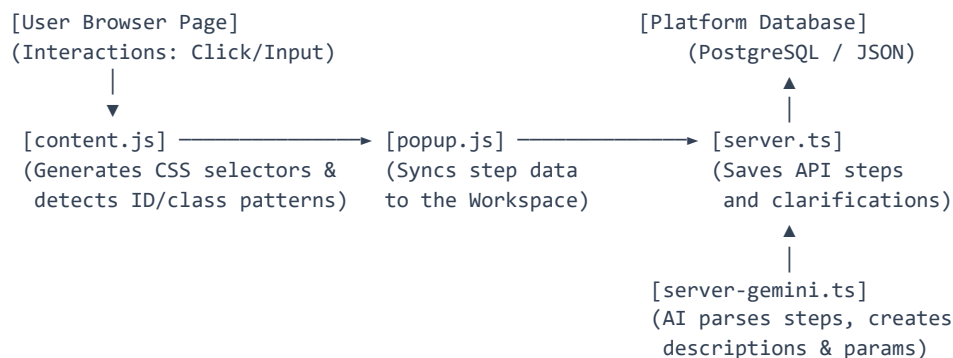


Shamsu Project Architecture Report

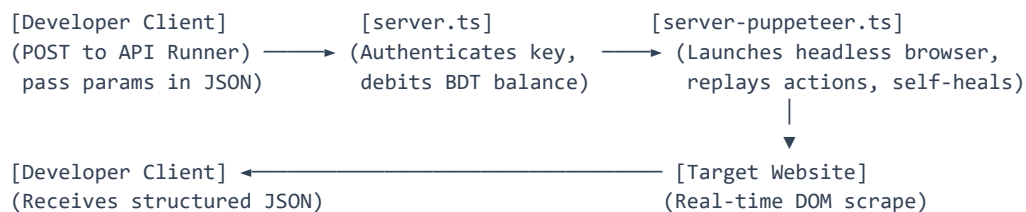
System Build Version: 3.15 (All Phases Complete)

This document details the exact recording, analysis, storage, replay, and recovery workflows implemented in the Shamsu (Browser API Recorder & Marketplace) codebase.

=== LIFECYCLE 1: RECORDING & API GENERATION ===



=== LIFECYCLE 2: CLIENT CALL & REPLAY ===



QUESTION 1 — Recording & API Generation

a) Capture Mechanism

Shamsu does **not** use proxy logs, DevTools network protocols, or service workers. It tracks DOM interactions directly in the active tab context. The extension injects event listeners using capture-phase event handling:

- `document.addEventListener("click", ..., true)` : captures mouse selection.
- `document.addEventListener("change", ..., true)` : captures text inputs and selects.

b) Captured Attributes

When an action triggers, Shamsu saves a structured step payload:

- `action` : "navigate" , "click" , "input" , or "scrape" .
- `url` : The web page address where the interaction took place.
- `selector` : The computed CSS selector targeting the node.

- `value` : Entered text for inputs.
- `description` : Descriptive logging string.

The selector generator strips dynamic classes (alphanumeric hashes of length 8-12) to avoid selector breakdown, falling back to unique attributes such as `data-testid` , `name` , or structural paths.

c) API Registration & Storage

Steps are sent from the extension back to the frontend workspace. A request is made to `/api/recorder/clarify` , where Llama 3.3 / Gemini analyzes the sequence and generates human documentation and dynamic parameter definitions. The result is saved via `/api/apis/create` to the remote Neon PostgreSQL database (the `datastore` table) or the local `data-store.json` backup.

d) Dynamic Arguments & Auth Recovery

Dynamic variables are marked with double curly braces (e.g. `{{search}}`). The Puppeteer replayer substitutes these at execution time. Expired logins are avoided because Shamsu replays the login script steps in the browser from scratch on every invocation.

e) File and Function Index

- **DOM Capture:** `browser-api-recorder-extension/content.js`
- **Dashboard Sync:** `browser-api-recorder-extension/popup.js`
- **AI Extraction:** `server-gemini.ts` (functions: `clarifyRecordedApi` , `generateStepsWithLLM`)
- **Datastore Wrapper:** `server-state.ts`

QUESTION 2 — Replay & Results

a) Execution Replay Flow

When a buyer calls the REST API runner `POST /api/apis/run/:apiId` , the server validates credentials, debits BDT balance, and invokes the Puppeteer runner. The runner launches a headless Chrome instance with stealth spoofing to avoid anti-bot filters, processes each database step one by one, and returns a JSON payload.

b) Authentication Validation

Auth is validated by re-executing recorded login scripts or reusing cookies stored in the session context.

c) Example Response

```
{
  "data": [
    {
      "hotel_name": "Premium Grand Millennium",
      "price": "14,500 BDT",
      "rating": "4.8 / 5"
    }
  ]
}
```

d) Self-Healing & Error Reporting

If a CSS selector fails due to DOM changes, Shamsu activates a semantic healing engine. It scans the page for nodes containing target text or synonyms. If it still crashes, a debug screenshot is saved to [screenshot_error.png](#) , and a HTTP 500 error is returned.

e) Dashboard Analytics Logs

The console records each call log including date, status (success/failed), cost (BDT), parameter inputs, latency in ms, and response payload, rendered dynamically in the admin & user views.